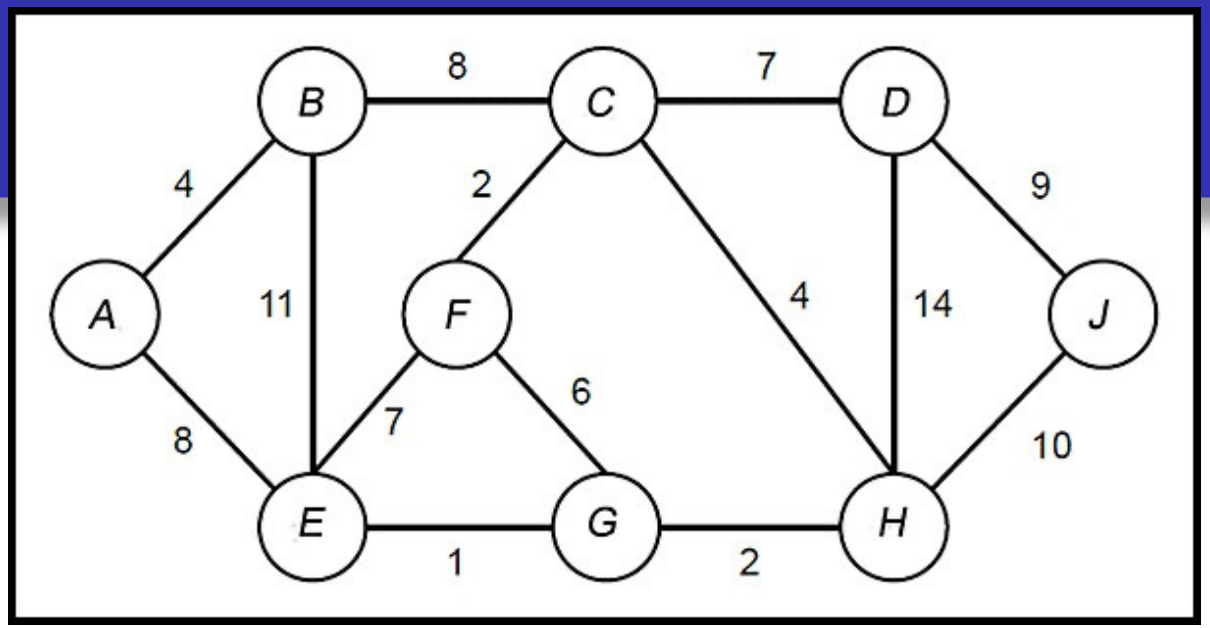
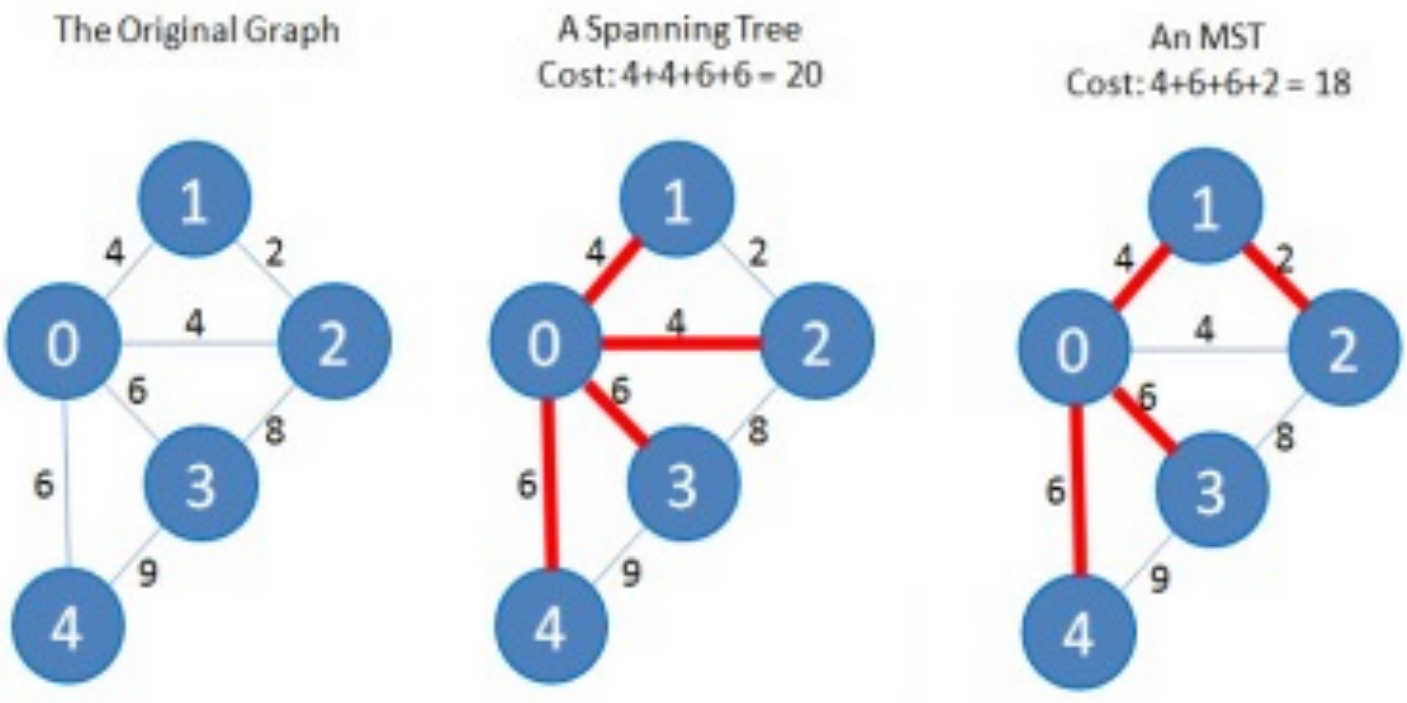
Minimum Spanning Tree



Adapted for ComO by Kamonphop Srisopha

Minimum Spanning Tree

- กำหนดให้ กราฟแบบมีน้ำหนัก G = (*V*,*E*) ที่ไม่มีทิศทางและเชื่อมกัน
- ปัญหานี้เลือก subset ของเส้นเชื่อม *E'* ของ *G* ที่ทำให้กราฟ *G* **ยังเชื่อมกัน**และน้ำหนักรวมของเส้นเชื่อมที่เลือก *E'* **น้อยที่สุด**



- ในการที่จะทำให้เงื่อนไขการเชื่อมกันสำเร็จนั้น
- เราต้องการอย่างน้อย ***V-1*** เส้นเชื่อมเพื่อที่จะทำให้เกิดโครงสร้าง tree และ tree นี้จะ**ต้องแผ่ทั่ว** (span) ทุกโหนด V ใน G ถึงจะเป็น spanning tree
- ถ้าหาแค่ spanning tree มีหลายอัลกอริทึมที่หาได้ ทั้ง DFS และ BFS ก็สร้าง spanning tree ได้
- แต่ท่ามกลาง spanning tree ทั้งหมด จะมีบางต้น(**อย่างน้อยหนึ่งต้น**) ที่มี**น้ำหนักรวมน้อยที่สุด (Minimum Spanning Tree)**

ปัญหา Minimum Spanning Tree(MST) มีหลาย application

- ตัวอย่างเช่น เราสามารถจำลอง**ปัญหาการสร้างถนนในเมืองที่ห่างไกล** โดยใช้ MST ได้ โดยที่**โหนดคือเมือง**และ**เส้นเชื่อมคือถนน**ที่เราอาจจะสร้างเพื่อเชื่อมระหว่างเมืองสองเมือง **ค่าใช้จ่ายในการสร้างถนน**เพื่อเชื่อมเมือง i และเมือง j คือน้ำหนักของเส้นเชื่อม (i, j) MST ของกราฟนี้คือค่าใช้จ่ายต่ำสุดในการสร้างถนนเพื่อให้เชื่อมทุกเมือง
- MST สามารถแก้ได้ด้วยหลายๆ algorithm เช่น Prim's และ Kruskal's ทั้งคู่เป็น **Greedy algorithm**
- น้ำหนักของ MST ที่ได้จากสองอัลกอริทึมนั้นเป็นค่าเฉพาะนั่นคือ อาจมี MST หลายต้นแต่มีค่าน้ำหนักน้อยสุดค่าเดียว

Minimum Spanning Tree

- ฉะนั้น, การที่จะเป็น Minimum Spanning Tree ต้อง

- 1. ทุก Vertices เชื่อมถึงกันหมด
- 2. ไม่มี Cycle
- 3. ค่าน้ำหนักรวมของเส้นเชื่อมต่ำที่สุด

Kruskal’s Algorithm

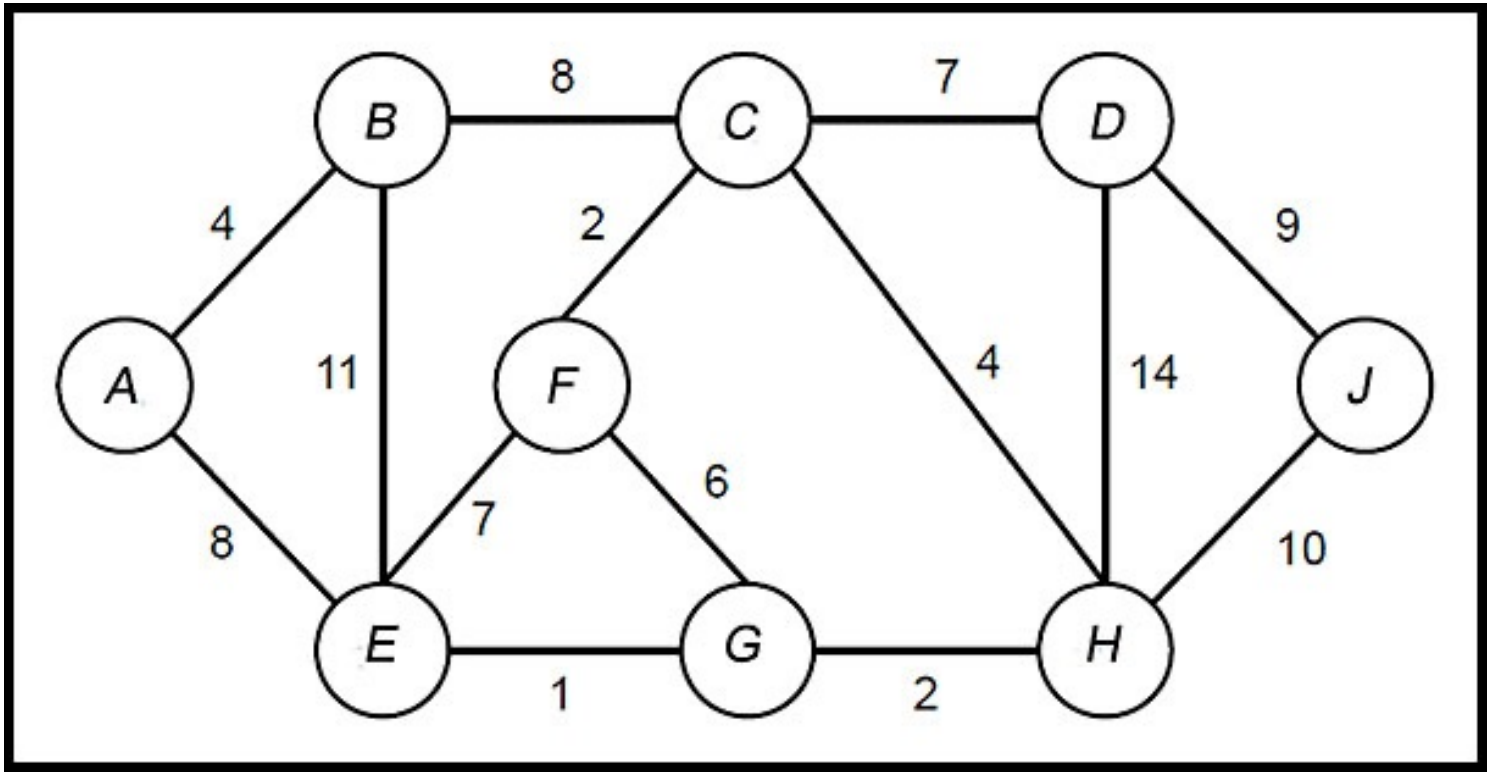
Kruskal's Algorithm

- Joseph Bernard *Kruskal* Jr.'s algorithm เริ่มต้นจะเรียงเส้นเชื่อม *E* ตามน้ำหนักจากน้อยไปมาก สามารถทำได้ไม่ยากโดยการเก็บเส้นเชื่อมไว้ใน **EdgeList** จากนั้น sort มันจากน้อยไปมาก Kruskal's algorithm จะลองเพิ่มเส้นเชื่อมเข้าไปใน MST ตราบเท่าที่การเพิ่มเส้นเชื่อมนั้น ไม่ทำให้เกิด cycle
- ทำไปรื่อยๆจนได้ edge ครบ V-1
- ในการตรวจสอบ cycle นั้นทำได้โดยการใช้ **Disjoint Set Union!!!** (เราใช้ DSU ตรวจสอบได้อย่างไร?)
- Disjoint Set Union? ทำงานอย่างไร?

- Code ค่อนข้างสั้นเพราะว่าเราแยกส่วน Union-Find Disjoint Sets ไปอีกฟังก์ชัน
- เวลาในการทำงานเป็น O(sorting+ trying to add each edge × cost of Union-Find operations) = $O(E \log E + E \times (\approx 1)) = O(E \log E) = O(E \log V^2) = O(2E \log V) = O(E \log V)$.

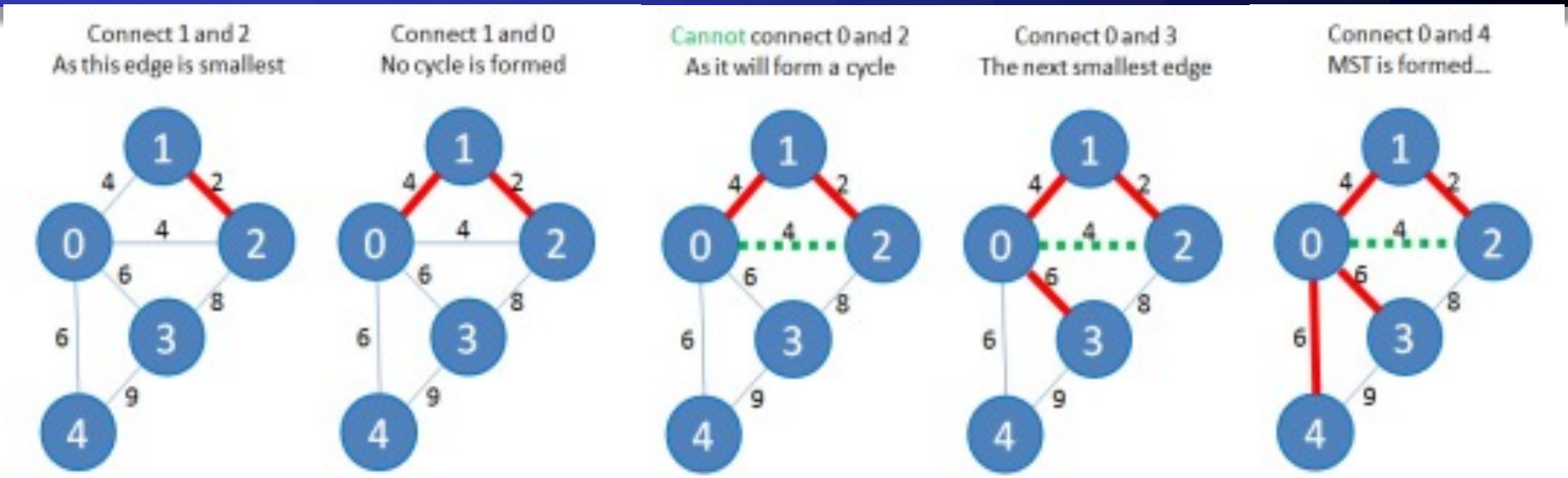
จาก DSU เพราะเร็วมาก
นับได้ว่าเป็น constant
time = (α(V))

จงหา MST จากกราฟด้านล่างโดยใช้ Kruskal's algorithm

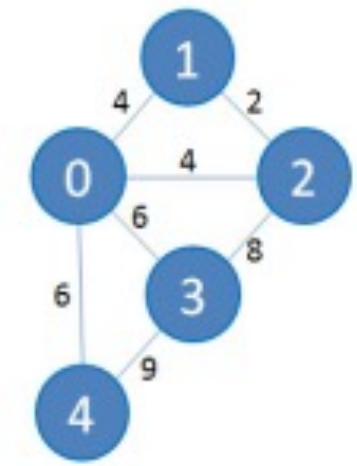


Exercise 1

- Let’s implement Kruskal’s algorithm



The Original Graph



5 7 #V, #E

0 1 4 e(a,b,w)

0 2 4

0 3 6

0 4 6

1 2 2

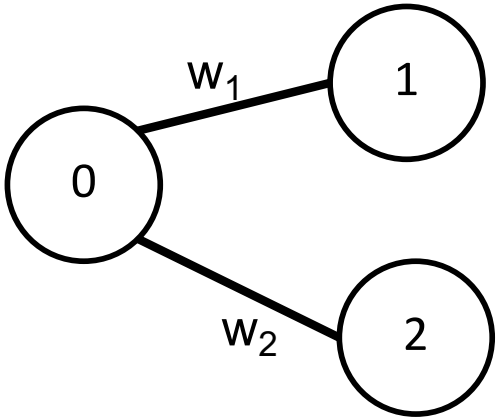
2 3 8

3 4 9

Prim’s algorithm = Pick a node

Prim's Algorithm

- Robert Clay Prim's algorithm นั้นเริ่มจากเลือก start node หนึ่งตัว ให้เป็น node v
- จากนั้น mark ว่า v ถูกเลือกเข้า MST แล้ว (ตรงนี้จะทำเป็น set ของ node ก็ได้ = เรียกว่า visited)
- แล้วนำ edge ทุกเส้นจาก v ไปยัง node อื่นๆ ที่ยังไม่ถูกเลือกเข้า MST ใส่ลงใน priority queue ในรูป (w, u) โดย w คือค่าน้ำหนักของ edge



visited = {0}

pq =[(w1,1), (w2, 2)]

*assume $w_1 <= w_2$

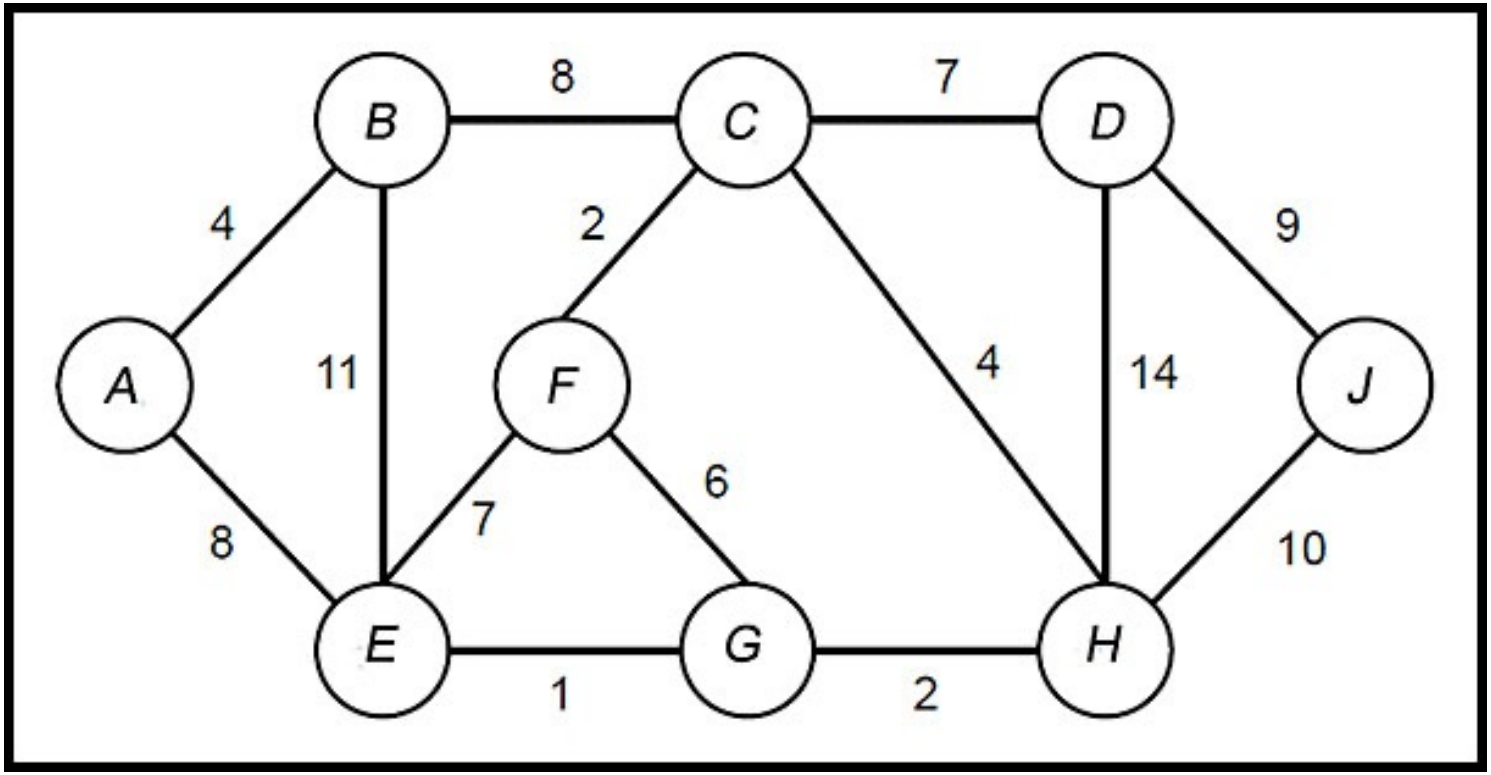
- ข้อมูลใน priority queue จะถูกเรียงตามน้ำหนักของ edge จากน้อยไปมาก และถ้าน้ำหนักเท่ากันอาจเรียงต่อด้วยหมายเลข node เพื่อ break tie
- จากนั้น algorithm จะเลือก edge ที่มีน้ำหนักน้อยที่สุดออกจาก priority queue
- ถ้า node ปลายทางของ edge นั้นยังไม่ถูกเลือกเข้า MST (ยังไม่ได้อยู่ใน visited) เราจะรับ edge นี้เข้า MST, บวก น้ำหนักของ edge นั้น เข้ากับค่าน้ำหนักรวมของ MST, และ mark u ว่าถูกเลือกแล้ว (add เข้าไปใน visited)
- แล้วนำ edge ทุกเส้นจาก u ไป node อื่นที่ยังไม่ visited ใส่เข้าไปใน PQ

- ทำซ้ำไปเรื่อย ๆ จน priority queue ว่าง หรือจนมี node ถูกเลือกครบทุก node

- ความซับซ้อนของวิธีนี้คือ O(E log E) และมักเขียนย่อเป็น O(E log V) เพราะ จำนวน E <= V^2

- พิจารณา O(E) edge และแต่ละ edge มีการ push (O(log E)) และ pop (O(log E)) จาก PQ เพราะฉะนั้นจะได้ O(E log E)

จงหา MST จากกราฟด้านล่างโดยใช้ Prim's algorithm



Exercise 2

- Let’s implement Prim’s algorithm

ตัวอย่างกราฟ

5 7

0 1 4

0 2 4

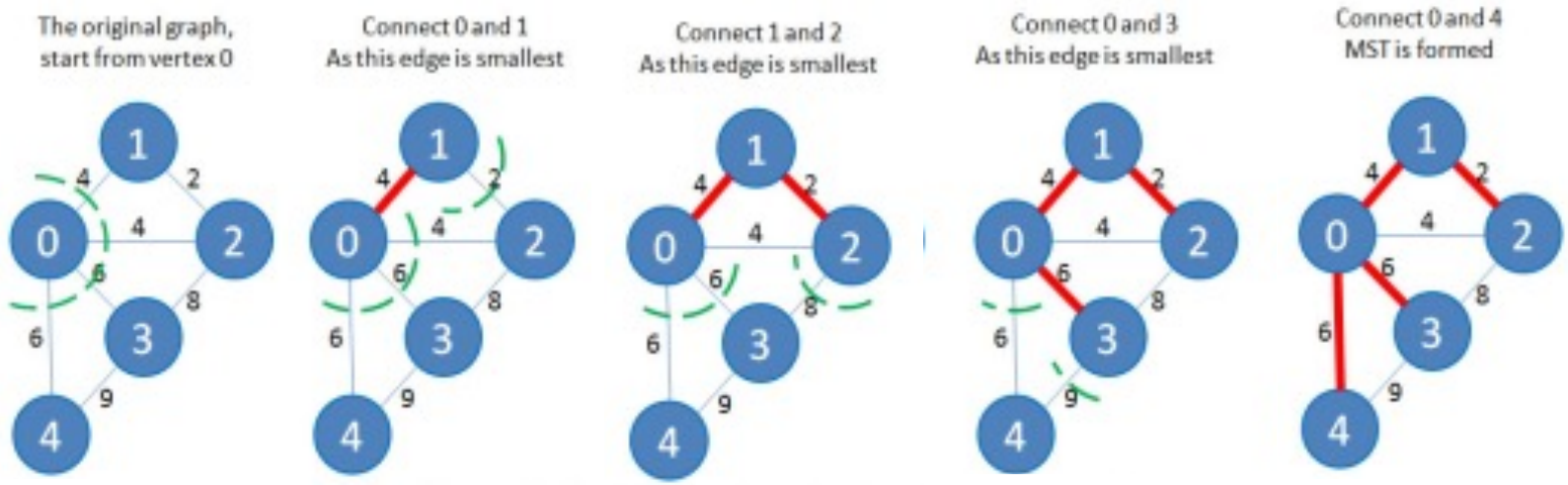
0 3 6

0 4 6

1 2 2

2 3 8

3 4 9



ทดสอบความเข้าใจ

Q1

จริง หรือ ไม่

- กราฟที่มีค่าน้ำหนักบนเส้นเชื่อม จะมี MST ที่เป็นเอกลักษณ์ ได้ก็ต่อเมื่อค่าน้ำหนักของเส้นเชื่อมแต่ละเส้นแตกต่างกันทั้งหมด

Q2

จริง หรือ ไม่

- กำหนดกราฟ G มาให้ ถ้าเส้นเชื่อม e ไม่ได้เป็นส่วนหนึ่งของ MST ใด ๆ ของ G เลย ฉะนั้น e จะต้องเป็นเส้นเชื่อมที่มีน้ำหนักมากที่สุดบน cycle ใด cycle หนึ่งใน G

Q3

จริง หรือ ไม่

- กำหนดกราฟถ่วงน้ำหนักที่เชื่อมกัน G = (V,E,w) มาให้ สมมุติเพิ่มอีกว่า |E| + 1 > |V| และมีเส้นเชื่อมหนึ่งเส้น e ที่มี w(e) > w(e′) สำหรับทุก e′ ∈ E และ e′ != e (กล่าวคือน้ำหนักเส้น e มากที่สุด) เราจะสามารถสรุปได้ว่า e จะต้องไม่อยู่ใน MST อย่างแน่นอน

Q4

- กำหนดให้ v เป็น vertex หนึ่งในกราฟถ่วงน้ำหนักที่เชื่อมถึงกัน (weighted connected graph) G=(V,E,w)
- จงอธิบาย algorithm สำหรับค้นหา Minimum Spanning Tree (MST) ของกราฟ G ที่ทำให้ดีกรี (degree) ของ v มีค่าน้อยที่สุด (กล่าวอีกนัยหนึ่งคือ เป็น MST ที่มีจำนวนเส้นเชื่อมมาต่อกับจุดยอด v น้อยที่สุดเท่าที่จะเป็นไปได้ เมื่อเทียบกับบรรดา MST ทั้งหมดที่สามารถสร้างได้จากกราฟ G)

Q5

- กำหนดกราฟถ่วงน้ำหนักแบบเชื่อมกัน G = (V, E, w) จงอธิบาย algorithm ในการหา Maximum Spanning Tree ของกราฟ G โดยให้มี time complexity เป็น O(E log V)

Q3

จริง หรือ ไม่

- Kruskal's algorithm ไม่สามารถทำงานได้ถ้าเจอกราฟที่มีน้ำหนักของเส้นเชื่อมเป็นค่าลบ
- Prim's algorithm ไม่สามารถทำงานได้ถ้าเจอกราฟที่มีน้ำหนักของเส้นเชื่อมเป็นค่าลบ